TASK 2: Choose the Data Representation

Name: K.V.N.D.P.SAI RUPESH | Register Number: 1972260102 | Java Fundamentals and OOP - Assignment 1

Data types and why

Data	Java type	Why
Slot number, floor number	int	Small whole numbers that are used as array indexes
Vehicle (registration) number	String	Letters and digits, and it must be compared with equals()
Entry and exit time	int (minutes after midnight, 0 to 1439)	Simple arithmetic for duration; a long timestamp would suit a real system
Duration	int (minutes)	Whole minutes; fees are based on started hours
Fee	double	Whole-rupee amounts here. A real billing system would use BigDecimal, because double can round 0.1 incorrectly
Charging units (kWh)	int	Billed in whole units

Floors and slots: jagged array

The facility has floors with different numbers of slots (for example 2, 3, 1, 4, 2). A rectangular array such as new ParkingTicket[5][4] would create 20 slots, which is wrong and wastes memory. A jagged array is the right choice: the outer array holds one reference per floor, and each floor is an array of its own length.

```
ParkingTicket[][] slots = new ParkingTicket[slotsPerFloor.length][];
for (int f = 0; f < slotsPerFloor.length; f++)
    slots[f] = new ParkingTicket[slotsPerFloor[f]];
```

Where casting and type conversion are needed

Fees are charged per started hour, so minutes must be turned into hours. This is where precision is lost.

```
int minutes = 130;
int wrong   = minutes / 60;           // 2  (integer division drops the remainder)
int correct = (minutes + 59) / 60;    // 3  (every started hour is counted)
double frac = (double) minutes / 60; // 2.1666... (only if a fraction is needed)
```

Example of a wrong fee: a car parked for 130 minutes. Using wrong = 2 gives 2 x Rs 30 = Rs 60, which loses the last 10 minutes. The correct fee is 3 x Rs 30 = Rs 90. Our design avoids this by using (minutes + 59) / 60, so any part of an hour is charged as a full hour. Writing 130 / 60 * 30.0 would also fail because the division happens first.

How input is read

Scanner is used for interactive input, because a parking attendant types entries one at a time. Command-line arguments are fixed when the program starts, so they suit one-off runs only. The tests in this assignment use direct method calls, which is the same as typing the values.

Alternative: one flat array or ArrayList of floors

A flat array with index = floor x slotsPerFloor + slot works only when every floor has the same size. For unequal floors, store a start offset for each floor (start[f] = total slots on earlier floors), then index = start[f] + slot. Locating a slot is still O(1), but there is an extra array of F offsets and the arithmetic is easy to get wrong. An ArrayList gives O(1) access to each floor with get(f), and each floor can have its own length, so no offset arithmetic is needed. Space: the flat array uses one reference per slot plus F offsets; the ArrayList uses the same slot references plus a little list overhead and a small array header per floor. The jagged array we use has the same cost as the ArrayList and is simpler than the flat version.